Personal Research Journal

Author: Yiming Lu

Header & Context

Timestamp & Entry Title: October 15, 2026 - Real-Time Cartographer-Based Vehicle Motion Estimation

Project Name/Code: F1TENTH Autonomous Vehicle - ROS2 Cartographer Velocity Estimation; workspace: `~/newf1_ws`

Current Hypothesis/Goal: Use the Cartographer `map -> base_link` pose to estimate the ego vehicle's planar speed, X/Y velocity components, movement direction, and heading in real time. The resulting motion estimate will later be compared with the VESC-based motor-speed estimate and used to evaluate whether sensor fusion is worthwhile.

Methodology & Execution

Experiment / Computational Setup

The F1TENTH platform was operated with ROS2 Humble, a Hokuyo LiDAR, the VESC controller and its IMU, and Cartographer. The LiDAR stream `/scan` was verified at approximately 10 Hz and the IMU stream `/sensors/imu/raw` at approximately 50 Hz. Static transforms from `base_link` to `laser` and `imu` were active through `robot.launch.py`. Cartographer then produced the dynamic vehicle pose through the `map -> base_link` transform.

SYSTEM FLOW

```
Hokuyo LiDAR (/scan, ~10 Hz) -----\
                                    > Cartographer -> map -> base_link
VESC IMU (/sensors/imu/raw, ~50 Hz) -/                 |
                                                       v
                                          vehicle_state_estimator
                                          speed, vx, vy,
                                          movement direction,
                                          heading
```

Exact Code, Scripts, or Commands

BASH

```
$ source ~/newf1_ws/install/setup.bash
$ ros2 topic hz /sensors/imu/raw
$ ros2 topic hz /scan
$ ros2 launch my_robot_bringup robot.launch.py
$ ros2 launch my_cartographer_config cartographer.launch.py
$ ros2 run tf2_ros tf2_echo map base_link
$ ros2 pkg create --build-type ament_python vehicle_state_estimator \
    --dependencies rclpy tf2_ros std_msgs
$ colcon build --packages-select vehicle_state_estimator --symlink-install
$ ros2 run vehicle_state_estimator cartographer_velocity
```

Detailed Steps Taken

1. Sensor and USB identification. After powering the vehicle, `/dev/serial/by-id/` showed the Hokuyo LiDAR on `/dev/ttyACM0` and the STMicroelectronics ChibiOS VESC interface on `/dev/ttyACM1`. This check was repeated because the device numbering had changed after previous power cycles.

2. Sensor-rate validation. The VESC driver connected to firmware 6.6 and the IMU stream was measured at approximately 50 Hz. The Hokuyo driver streamed at approximately 9.96-10.00 Hz. These two sensor

streams are the live inputs used by the current Cartographer configuration.

3. Cartographer pose validation. After launch, `tf2_echo map base_link` produced continuously updating translation and rotation data. This established that Cartographer could supply time-varying vehicle position `(x, y)` and orientation for the velocity estimator.

Implementation of the Real-Time Velocity Estimator

ROS2 Package and Published Signals

A new Python package, `vehicle_state_estimator`, was created. The node `cartographer_velocity.py` listens to the TF tree, obtains the latest `map -> base_link` transform, and publishes five real-time scalar topics:

OUTPUT TOPICS

```
/cartographer_velocity/speed
/cartographer_velocity/vx
/cartographer_velocity/vy
/cartographer_velocity/movement_direction
/cartographer_velocity/heading
```

Velocity Calculation

The node does not read VESC motor speed for this estimate. Instead, it differentiates the Cartographer pose over time. For two pose samples separated by `dt`, the map-frame velocity components are calculated as `vx = (x2 - x1) / dt` and `vy = (y2 - y1) / dt`. Speed is `sqrt(vx^2 + vy^2)`. Movement direction is `atan2(vy, vx)`, converted to degrees, while heading is obtained from the yaw angle of the Cartographer quaternion.

PYTHON

```
vx = (x - old_x) / dt
vy = (y - old_y) / dt
speed = math.sqrt(vx * vx + vy * vy)
movement_direction_deg = math.degrees(math.atan2(vy, vx))
heading_deg = math.degrees(yaw)
```

Noise Filtering and Parameter Tuning

Direct numerical differentiation amplifies small localization jitter. An initial 0.5 s window with a 0.01 m/s deadband still produced occasional stationary values near 0.010-0.020 m/s. Increasing the deadband to 0.025 m/s removed those stationary spikes, but it also suppressed a deliberately slow push whose average speed was approximately 0.0184 m/s. The estimator was therefore changed to a 1.0 s history window and a 0.005 m/s deadband. After a clean sensor and Cartographer restart, the stationary output returned to 0.0 while slow translational motion remained detectable.

PARAMETER UPDATE

```
# initial test
window_time = 0.5 s
deadband    = 0.010 m/s

# stationary-noise suppression test
deadband    = 0.025 m/s

# final working test configuration
window_time = 1.0 s
deadband    = 0.005 m/s
```

Real-Time Motion Test

With the car kept on the floor and translated manually, the speed topic rose from zero, changed continuously during motion, and returned to zero after stopping. Representative speed samples included approximately 0.10-0.13 m/s and a peak near 0.186 m/s. The X component reached a similar magnitude, while the Y component was generally smaller, indicating that the tested motion was dominated by one map axis.

REPRESENTATIVE OUTPUT

```
speed: 0.1158, 0.1264, 0.1275, 0.1859 m/s
vx:   -0.1156, -0.1263, -0.1274, -0.1856 m/s
vy:    values mostly near a few hundredths of m/s

stationary after restart:
speed = 0.0 m/s
```

Results & Observations

Measurement / condition	Observed result	Interpretation
VESC IMU input	~50 Hz	Stable after VESC connection restored
Hokuyo LiDAR input	~9.96-10.00 Hz	Stable
Cartographer pose	map -> base_link available	Position and orientation update in real time
Stationary speed	0.0 m/s after final restart/tuning	Deadband/window suppress small pose jitter
Manual-motion speed	Up to ~0.186 m/s observed	Real-time response confirmed
Velocity X / Y	vx dominant; vy smaller	Direction components respond to map-frame motion
VESC vs Cartographer comparison	Not completed yet	Requires synchronized measurement

Raw Data & Outputs

OUTPUT

```
# IMU
average rate: ~49.99-50.00 Hz

# LiDAR
average rate: ~9.96-10.00 Hz

# Cartographer velocity topics
/cartographer_velocity/speed
/cartographer_velocity/vx
/cartographer_velocity/vy
/cartographer_velocity/movement_direction
/cartographer_velocity/heading

# Example real-time speed
0.0734, 0.1158, 0.1264, 0.1275, 0.1682, 0.1859 m/s
```

Movement Direction and Heading

The movement-direction topic and heading topic were both shown to update in real time. One recorded movement-direction sequence was near -72.55 degrees, while a separately recorded heading sequence changed from approximately -13 degrees toward -16 degrees. These two sequences should not yet be interpreted as a valid same-time angular error because the vehicle had been repositioned before the test and the two values were not logged synchronously. A synchronized terminal logger was therefore planned so that speed, Vx, Vy, movement direction, heading, and their angular difference could be inspected from the same timestamped calculation.

Unexpected Behavior

The most important interruption occurred when the VESC driver disappeared from the ROS graph. `/sensors/imu/raw` then had a publisher count of zero, Cartographer no longer created the `map` frame, and the velocity node stopped publishing because `map -> base_link` was unavailable. Linux still detected the STMicroelectronics Virtual COM Port through `lsusb`, but `/dev/ttyACM1` temporarily disappeared. Kernel logs showed USB disconnect events followed by `cdc_acm ... ttyACM1: USB ACM device` when the connection returned. Restarting the VESC driver restored the 50 Hz IMU stream; Cartographer and the velocity estimator were then restarted cleanly.

DIAGNOSTIC COMMANDS

```
$ ros2 node list | grep vesc
$ ros2 topic info /sensors/imu/raw --verbose
$ ls -l /dev/serial/by-id/
$ lsusb | grep -i -E "STMicro|ChibiOS"
$ sudo dmesg | tail -n 40

# recovered kernel binding
cdc_acm 1-2.3:1.0: ttyACM1: USB ACM device
```

Synchronized Logger Issue

An edit was made to print synchronized motion values directly from the velocity node. The first edit caused a Python `NameError` because `movement_direction_deg` was referenced inside the helper function `publish_heading()`, where that variable did not exist. A corrective edit was prepared to restore `publish_heading()` to heading-only publication and keep the synchronized calculation inside the main timer callback. Final validation of this synchronized terminal output remains pending.

Analysis & Reflection

Data Interpretation

The experiment demonstrated that vehicle translation can be estimated directly from the Cartographer pose rather than from motor rotation. This distinction is important: the earlier VESC method estimates longitudinal vehicle speed from motor ERPM, while the present method measures change in the estimated vehicle pose in the map frame. As a result, the Cartographer method can provide a two-dimensional velocity vector and movement direction in addition to speed magnitude.

The current estimate should still be described as a pose-derived velocity estimate, not ground-truth speed. Localization noise, scan-matching corrections, sensor dropouts, and the choice of differentiation window can all affect the result. The 1.0 s window improves stability but also smooths rapid speed changes, so the filter parameters may need further tuning for realistic F1TENTH driving speeds.

Comparison with the VESC Method

The two methods provide complementary information. VESC-derived speed is high-rate and directly related to drivetrain rotation, but it can disagree with ground motion during wheel slip and still depends on an uncalibrated ERPM-to-m/s gain. Cartographer-derived motion is based on the estimated vehicle displacement in the environment and therefore provides vx, vy, movement direction, and heading.

However, pose differentiation is more sensitive to localization jitter and map corrections. A synchronized comparison has not yet been completed, so neither source should currently be treated as the definitive ground-truth velocity.

Sensor-Fusion Feasibility

The results support further investigation of sensor fusion. A future estimator could use fast VESC longitudinal velocity as a short-term motion input while using Cartographer position and heading to correct accumulated drivetrain error. A ROS2 state-estimation framework such as an EKF can be evaluated after both sources are synchronized and individually calibrated. The present work establishes the signals required for that comparison but does not yet implement fusion.

Next Steps & Open Questions

Immediate Action Items

1. Complete and validate the synchronized logger so that `Speed, Vx, Vy, Movement Direction, Heading`, and the difference between movement direction and heading are printed from the same calculation cycle.

2. Perform a controlled ground test without lifting or reorienting the vehicle during recording: first remain stationary, then translate straight forward, then rotate in place. This will separate heading change from translational movement and provide a clean directional validation.

3. Record Cartographer-based speed and VESC-based odometry at the same time and compare both curves during controlled motion. The VESC conversion factor should be calibrated before treating its magnitude as accurate.

4. Quantify stationary drift, response delay, and filtering trade-offs for different history-window lengths and deadbands.

5. If both independent estimates are sufficiently stable, evaluate a ROS2 EKF / `robot_localization` configuration for fused vehicle state estimation.

Unresolved Questions

The remaining questions are how stable the Cartographer velocity estimate remains at normal F1TENTH driving speeds; what time window gives the best compromise between noise rejection and response delay; how closely VESC-derived and Cartographer-derived speed agree after calibration; whether movement direction follows heading during straight motion when measured synchronously; and whether USB reliability needs a more robust connection strategy before repeated experiments.

Conclusion

This week extended the project from motor-based speed estimation to real-time Cartographer-based vehicle motion estimation. The system now publishes speed, map-frame X/Y velocity, movement direction, and heading from the `map -> base_link` pose. Stationary and translational tests confirmed real-time response after filter tuning, and a USB/IMU interruption was diagnosed and recovered. The next milestone is a synchronized comparison between Cartographer and VESC velocity, followed by an evaluation of sensor fusion.